Understanding Context, Memory and Conversational Branching

🧠 What "Context" Means in Gemini CLI

Think of **context** as Gemini's *temporary brain* — kind of like your short-term memory.

It remembers what's happening **right now** in your current chat or task.
But if you shut it down, restart it, or type `/clear`, that memory is *gone*.
That's why it's called **volatile** — like RAM in a computer: fast, but forgetful once turned off.

💡 Example:

Let's say you open Gemini CLI and chat like this:

```
You: Help me write a Python script for sending emails.
Gemini: Sure, here's the code…
You: Now make it send attachments too.
```

Gemini knows you're talking about the same script because both your messages exist **in its current context**.
If you type `/clear` and then say:

```
You: Make it send attachments too.
```

Gemini will be confused — it won't know what script you're talking about because that "context" (the past messages) is gone.

🧱 How Context is Built (Like Layers)

When Gemini CLI starts up, it fills its short-term memory with a few basic things:

1. **System Instructions** – These are like the ground rules it follows.
   (e.g., how it should respond, tone, or default commands.)

2. **GEMINI.md files** – These are like mini-guides or notes the CLI reads to understand your project better.

3. **Your Project Folder** – Gemini automatically "notices" what files exist.
    - If it sees a `package.json`, it knows you're working on a Node.js project.
    - If it sees `requirements.txt`, it assumes Python.
    - This helps it give more accurate answers.
4. **Conversation History** – Everything you and Gemini say during a session adds more context.
   That's how it keeps up with follow-up questions.
5. **Extras You Add During the Chat:**
    - `@filename` → Gemini loads the **contents** of that file into memory (so it can read and use it).
    - `!command` → If you run a shell command, Gemini remembers the **output** too.

🧩 Why Managing Context Matters

Gemini can handle up to **1 million tokens** (that's a *lot* — like a whole book's worth of text).
But more isn't always better.

If you feed it too much random or outdated stuff, it can get **confused** — this is called **context bloat**.
So you need to manage it sometimes.

⚙️ Commands That Control Context

`/clear` → Hard Reset

Wipes everything.
Like restarting your brain — no memories of the chat.
Use when you want to start a *totally fresh topic.*

🧠 Example:
You were debugging code, but now want to write a poem.
Instead of carrying all that coding talk forward, you type `/clear` so Gemini doesn't mix topics.

`/compress` → **Smart Summary**

This command tells Gemini:

> "Hey, summarize everything we've discussed so far so we can free up memory but keep the main ideas."

It'll replace your huge chat history with a short summary.
This keeps Gemini focused but still "aware" of what's happened.

🧠 Example:
You've had a 200-line conversation about building a website.
Instead of keeping all that, `/compress` might save a short version like:

> "We're building a portfolio website using React and Firebase. The next step is adding user login."

Now Gemini has more space to continue without forgetting the key goal.

⚠️ Just be careful — summaries can lose *details*.
If you're working on complex logic or long code, `/compress` might accidentally skip something important.

So basically:

Command	What It Does	When to Use
`/clear`	Wipes all memory	When you want a fresh start
`/compress`	Summarizes old chat to save space	When the chat gets long but you want to keep the gist

🧠Conversational Branching

When you're using **Gemini CLI**, you're not always doing *one* thing.
You might be:

- coding a backend API,
- fixing bugs in your frontend,
- testing something new,

- or just chatting about cricket (like in the example).

Now imagine if Gemini *forgot everything* every time you switched topics — that'd be a nightmare.
That's where **conversation branching** steps in.

It lets you **save**, **pause**, and **resume** multiple conversations — like tabs in a browser, but for Gemini.

💬 Why You Need This

In real life, devs don't work linearly. You're constantly:

- jumping between tasks,
- pausing something midway,
- researching something else,
- and coming back later.

If you did that *without branching*, Gemini's memory would get all jumbled up — mixing cricket stats with your backend debugging 🧩😅

Branching keeps things clean and organized.
Each topic stays in its **own lane**.

⚙️ The Tools — `/chat` Commands

Gemini gives you a few commands to manage these "conversation branches":

Command	What It Does	Example
`/chat save <tag>`	Saves your current chat with a name (the `<tag>`)	`/chat save cricket-chat`
`/chat resume <tag>`	Reopens a saved chat (loads all its context again)	`/chat resume cricket-chat`
`/chat list`	Shows all saved chats	`/chat list`

`/chat delete <tag>`	Deletes a saved chat	`/chat delete cricket-chat`
`/clear`	Clears current screen + current session (starts fresh)	`/clear`

🧩 Example: Switching Between Two Topics

Let's walk through this like a real workflow 👇

🏏 1. Start Talking About Cricket

You're asking Gemini about the best batsmen:

```
You: Who are the top 5 test batsmen ever?
Gemini: (lists names)
You: Tell me about Sir Vivian Richards.
```

You've got a nice little conversation going.
Now you want to **pause** this and switch topics.

So you save it:

```
/chat save cricket-test-batsmen
```

✅ Gemini stores your entire conversation — all questions, answers, and context — under the name **cricket-test-batsmen**.

🧼 2. Clear the Screen

Now you want a clean slate:

```
/clear
```

Your screen and context are wiped.
Gemini forgets about the cricket talk (temporarily).

💻 3. Start a New Chat About Coding

You switch gears:

```
You: Help me create a study plan for learning the Agent Development
Kit.
Gemini: (creates a plan)
```

You also want to save this one:

```
/chat save adk-study-plan
```

✅ Now you've got **two separate saved conversations**.

📋 4. View Saved Chats

```
/chat list
```

Gemini shows:

```
1. cricket-test-batsmen
2. adk-study-plan
```

So you know what's available to resume.

🔁 5. Switch Back Anytime

If you want to jump back to cricket mode:

```
/clear
/chat resume cricket-test-batsmen
```

Boom — Gemini loads everything you'd said earlier.
It's like reopening a paused tab — it remembers Sir Viv, the list of players, everything.

You can now continue seamlessly:

```
You: Tell me about Sachin Tendulkar too.
```

🧹 6. Delete Old Chats (Optional)

When you don't need a chat anymore:

```
/chat delete cricket-test-batsmen
```

Poof 💨 — it's gone from your list.

🗂️ Where These Chats Are Stored

Gemini saves them in your system folders (automatically):

- **Windows:**

  ```
  C:\Users\<YourUsername>\AppData\Roaming\google-generative-ai\checkpoints\
  ```

- **Mac/Linux:**
  `~/.config/google-generative-ai/checkpoints/`

When you run `/chat list`, Gemini looks into those folders to see which chat "checkpoints" exist.

🧠 Quick Analogy

Think of Gemini like a game with multiple save slots 🎮

You're Doing	Gemini Command	What Happens
Save your progress	`/chat save project1`	New save slot created
Load your progress	`/chat resume project1`	Resume where you left off
Check your saves	`/chat list`	See all save slots
Delete a save	`/chat delete project1`	Removes that slot

Start a new game	`/clear`	New blank game

🔄 TL;DR — Conversational Branching

- It's like **multi-tasking for chats**.
- Keeps your work organized and prevents context confusion.
- Use `/chat save` and `/chat resume` to switch between topics anytime.
- Stored locally, so even after closing Gemini CLI, your chats are safe.

🧠 Gemini CLI's Memory — The Persistent Brain

So, **context** is like your brain's *working memory* — it only remembers what's happening *right now*.
But **memory** is more like *long-term knowledge* — the stuff you've learned and remember even after sleeping or restarting your computer.

That's exactly how Gemini CLI works.
Memory is what helps it *stay consistent* across sessions — even after you shut it down or move between projects.

🏗️ Two Types of Memory

Gemini CLI has **two main types** of memory:

1. **Instructional Memory** → Long-term *behavioral rules* (how Gemini should act)
2. **Factual Memory** → Long-term *facts* or preferences (what Gemini should remember about *you* or your project)

🧩 1. Instructional Memory — The GEMINI.md Files

Think of **GEMINI.md** as Gemini's *instruction manual* — a file where you tell it:

> “Here’s who you are, how to behave, and how to help me.”

It’s written in Markdown (`.md`) and can contain:

- Rules (e.g., “Always write clean code with comments”)
- Style guides (e.g., “Use TypeScript instead of JavaScript”)
- Project info (e.g., “This is a Django backend project”)
- Persona tweaks (e.g., “Act like a senior developer mentor”)

📚 How It Works — The Hierarchy

Gemini doesn’t just read one file — it *layers* multiple GEMINI.md files based on *where you are* in your system.

It’s like an onion 🧅 — different layers, from global to specific.

Level	Location	Purpose
🌍 **Global Context**	`~/.gemini/GEMINI.md`	Rules for *everything* (applies across all projects)
🏗️ **Project / Ancestor Context**	Any GEMINI.md in parent folders	Rules for your *current project*
🧩 **Sub-directory Context**	GEMINI.md in subfolders	Rules for a *specific module or component*

💡 **Example:**

- You have a *global* rule saying “Always use TypeScript.”
- Your *project* rule says “Use JavaScript for this project.”
  → Gemini will follow the project’s rule because it’s *more specific.*

That’s the power of the **hierarchy** — local rules override global ones.

⚙️ Commands for Managing Instructional Memory

Command	What It Does	Example

`/memory show`	Displays the *combined* instructions Gemini is currently using	`/memory show`
`/memory refresh`	Reloads all GEMINI.md files after you've made changes	`/memory refresh`

So if you tweak your rules in GEMINI.md, no need to restart the CLI — just refresh the memory.

🧠 Bonus: Modular Instructions with `@import`

You can also **split** your GEMINI.md into smaller parts for clarity.

Example:

```
@/path/to/team-guidelines.md
@/path/to/code-style.md
```

That way, your main GEMINI.md stays clean while Gemini still loads everything it needs.

🗒️ 2. Factual Memory — Storing Facts & Preferences

This is Gemini's "sticky note" memory — the stuff it remembers *about you*.

There are two ways to add these facts:

🧰 (a) The SaveMemory Tool

You can tell Gemini naturally, like:

> "Remember that my Google Project ID is `gcp-rocks-123`."
> "Remember to call me Ashna."
> "Remember I use Tailwind CSS for styling."

When Gemini hears that, it stores it *behind the scenes* in your global configuration.
This helps it remember small details to keep your experience consistent.

🧑‍💻 (b) The `/memory add` Command

If you want to do it *manually*, you can directly add facts with this command:

```
/memory add "My preferred CSS framework is Tailwind CSS."
```

This will literally **append** that line to your global file:

```
~/.gemini/GEMINI.md
```

Now, next time you open Gemini, it already knows your preferences.

🧠 Example Workflow

Let's go step-by-step:

Start Gemini CLI

```
gemini
```

1.

Add a preference

```
/memory add "My preferred CSS framework is Tailwind CSS."
```

2.

Check if it saved

```
/memory show
```

3. You'll see your Tailwind line there.

Use it

```
You: Generate a landing page using my preferred CSS framework.
```

4. Gemini should now use Tailwind automatically (if it recalls it properly).

⚠️ Note

Sometimes, users say Gemini doesn't always recall these added facts perfectly — it depends on how you phrase your prompt.

So, instead of:

> "Make a page."

Say:

> "Make a page using my saved CSS preference."

That helps Gemini "wake up" its factual memory better.

🗃️ Why This Matters

- 🧩 **GEMINI.md** = teaches *how Gemini should behave* (rules/personality)
- 🧠 **/memory add or SaveMemory** = teaches *what Gemini should remember* (facts/preferences)

Together, they make Gemini not just reactive, but **personalized and persistent**.

💾 Bonus Tip — Team Use

Since GEMINI.md is a simple markdown file, you can:

- Add it to Git for version control,
- Share it across your team,
- Standardize how all devs use Gemini.

Example:
Every dev in your company has a `GEMINI.md` that says:

> "Use TypeScript, prefer Jest for testing, and follow Airbnb code style."

Boom — consistent coding assistant for the entire org 💼🔥

🧩 TL;DR — Gemini CLI's Memory in One Table

Type	What It Stores	Where It Lives	Commands	Example
🧠 **Instructional Memory**	Rules & behavior	GEMINI.md (global, project, subfolder)	`/memory show`, `/memory refresh`	“Always use TypeScript.”
💾 **Factual Memory**	Facts & preferences	Appended to global GEMINI.md	`/memory add`	“My CSS framework is Tailwind.”

So basically:

> **Context** = short-term brain 🕰️
> **Branching** = multitasking brain 🔀
> **Memory** = long-term brain 💾

All three together make Gemini CLI feel less like a terminal app — and more like a *personal coding partner* that *actually remembers you*.

🧩*This Document is an easy version of the following article:*

https://medium.com/google-cloud/gemini-cli-tutorial-series-part-9-understanding-context-memory-and-conversational-branching-095feb3e5a43

Made By: Ashna Ghazanfar